Create LangChain Roadmap

review this and create a roadmap:
https://www.udemy.com/course/langchain/?couponCode=KEEPLEARNING

Yes. I reviewed the **exact Udemy course** you linked, including the current July 2026 version. It is now **19h 43m, 28 sections, 186 lectures**, rated **4.6/5 from 52K+ ratings**, and was re-recorded for **LangChain 1.2+**. Udemy

Open the Udemy course

My assessment for you

This is a very good course for your current AI-agent direction — but don't treat it as a "watch 20 hours of LangChain" course.

Given your existing work around **Ollama, Qdrant, SQL Server, .NET, semantic search and your Intelligent DB Agent**, I would use this course as the **learning backbone for building production-oriented agentic AI**, while replacing some of the course's examples with your own domain.

The course covers:

- LangChain fundamentals
- LCEL
- OpenAI + Ollama
- LangSmith
- Tools / tool calling
- Agents
- ReAct
- Structured output / Pydantic
- RAG
- Embeddings
- Vector databases
- Retrieval agents
- LangGraph
- MCP
- Deep agents
- Production-oriented RAG/agent architecture Udemy+1

The particularly important part for you is that the course explicitly moves from high-level `create_agent()` into understanding the underlying agent loop, tool calling and LangGraph architecture. Udemy+1

Recommended Roadmap

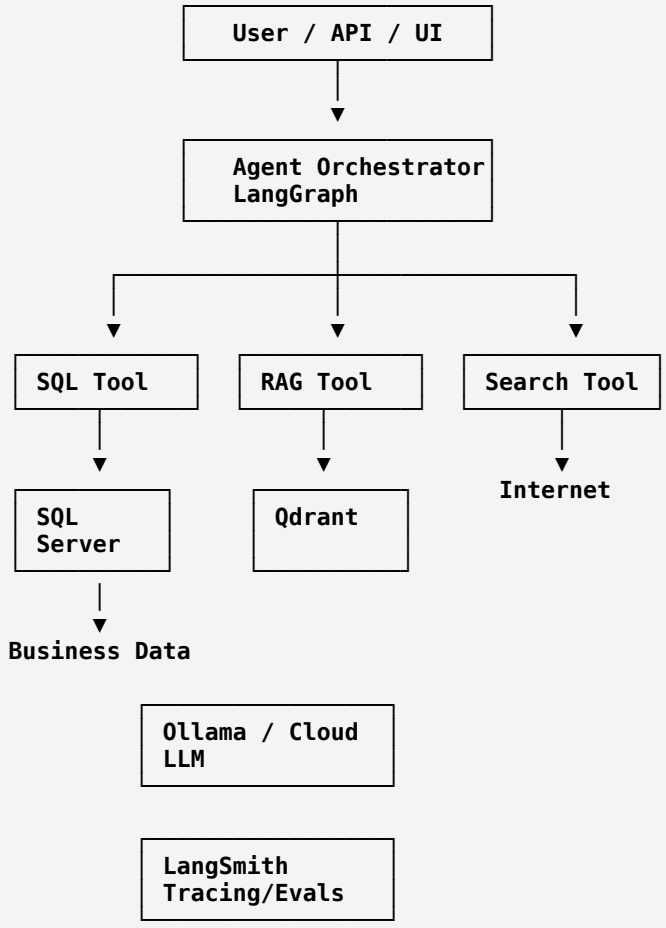
I would structure your learning into **8 phases over roughly 8–10 weeks**.

Don't aim to "finish the course."

Aim to finish with:

> **A production-style Intelligent Database + RAG + Agent platform.**

Your final architecture should look roughly like:



Phase 0 — Python AI Engineering Setup

Duration: 2–3 days

You already know software engineering, so **don't spend weeks learning Python**.

You only need enough Python to comfortably understand:

```
Python
class
function
```

```
decorator
async/await
typing
Pydantic
dataclass
dict/list
exceptions
environment variables
packages
virtual environments
```

The course itself assumes Python, Git, virtual environments, environment variables, classes, testing and debugging. Udemy

Learn

- **uv**
- virtual environments
- **pyproject.toml**
- **.env**
- Pydantic
- async Python
- basic FastAPI

Deliverable

Create:

```
ai-agent/
├── app/
│   ├── agents/
│   ├── tools/
│   ├── rag/
│   ├── models/
│   └── services/
├── tests/
├── .env
├── pyproject.toml
└── README.md
```

Phase 1 — LangChain Fundamentals

Duration: Week 1

Watch the beginning of the course carefully.

The course starts with:

```
Prompt
   ↓
Chat Model
   ↓
Chain
   ↓
Output Parser
```

and introduces LCEL:

```python
chain = prompt | llm | parser
```

The course specifically demonstrates prompt templates, chat models, chains, LCEL, debugging and model metadata. Udemy

Learn

Focus on:

- **`PromptTemplate`**
- **`ChatPromptTemplate`**
- Chat models
- **`AIMessage`**
- **`HumanMessage`**
- **`SystemMessage`**
- Output parsers
- LCEL
- **`.invoke()`**
- **`.stream()`**
- **`.batch()`**

Don't over-focus on

Old LangChain abstractions.

The ecosystem changes quickly.

Your mental model should be:

```
Input
 ↓
Prompt
 ↓
Model
 ↓
Structured Output
 ↓
Application Logic
```

Project

Build:

SQL Query Explanation Chain

Input:

```
Explain this SQL query and identify potential performance problems.
```

Output:

```JSON
{
  "summary": "...",
  "tables": [],
  "risks": [],
  "recommendations": []
}
```

This immediately connects the course to your existing SQL expertise.

Phase 2 — Models + Ollama

Duration: Week 1–2

This section is particularly relevant to you.

The course demonstrates switching from a cloud model to **Ollama** through LangChain while keeping the chain architecture essentially unchanged. Udemy

You should reproduce this with your local environment.

```
LangChain
    ├── OpenAI
    ├── Anthropic
    ├── Google
    └── Ollama
          └── Qwen / Mistral / Llama
```

Your experiment

Run the same task against:

```
Ollama
OpenAI
```

Compare:

Metric	Local	Cloud
Latency		
Accuracy		
Tool calling		
Structured output		
Cost		
Context handling		

This is much more valuable for you than simply watching the lecture.

Phase 3 — Tools + Agents

Duration: Week 2–3

This is where the course becomes highly relevant.

The course covers tools, ReAct, tool calling and the evolution toward modern LangChain agents. Udemy

Learn:

```
LLM
 ↓
Tool selection
 ↓
Tool execution
 ↓
Observation
 ↓
LLM
 ↓
Final answer
```

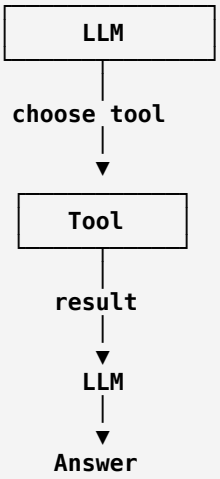
Understand the difference between:

Traditional chain

```
Input
 ↓
LLM
 ↓
Output
```

and:

Agent



The course also explains modern function/tool calling as structured machine-readable calls rather than relying solely on textual ReAct parsing. Udemy

Your project

Build:

Database Assistant Agent

Tools:

```
get_schema()
get_table_schema(table)
search_tables(keyword)
execute_readonly_sql(sql)
explain_sql(sql)
```

Agent:

```
User
 │
 ▼
Agent
 ├── get_schema
 ├── get_table_schema
 ├── generate SQL
 ├── validate SQL
 ├── execute SQL
 └── summarize result
```

This is basically the next evolution of your existing Intelligent DB Agent.

Phase 4 — Structured Output

Duration: 2–3 days

Do not skip this.

This is especially important for your applications.

The course demonstrates using Pydantic schemas with agents so that the model returns predictable structured data rather than arbitrary text. Udemy

For example:

```python
class SQLResponse(BaseModel):
    sql: str
    explanation: str
    tables_used: list[str]
    confidence: float
```

Then your application works with:

```
SQLResponse
```

instead of:

```
"Sure! Here's your SQL..."
```

Your rule

For business-critical systems:

> **LLM output → schema → validation → application**

not:

> **LLM output → directly execute**

Phase 5 — RAG + Qdrant

Duration: Week 3–5

This should be one of your biggest learning phases.

The course covers:

```
Documents
 ↓
Chunking
 ↓
Embeddings
 ↓
Vector DB
 ↓
Retriever
 ↓
LLM
```

It also covers the difference between naive retrieval and LCEL-based RAG, along with tracing and composability. Udemy

The course uses Pinecone in its examples, but **you should substitute Qdrant**, because you already use it.

Your architecture:

```
SQL Server
   │
   ▼
Extract business knowledge
   │
   ▼
Normalize
   │
   ▼
Chunk
   │
   ▼
Embeddings
   │
   ▼
Qdrant
```

Then:

```
User Question
      │
      ▼
Embedding
```

```
      |
      ▼
Qdrant
      |
      ▼
Relevant Documents
      |
      ▼
LLM
      |
      ▼
Grounded Answer
```

Phase 6 — Build Your Real RAG System

Duration: Week 5

Don't build another generic PDF chatbot.

Build:

Property Handover Knowledge Assistant

Use your actual domain.

Index things like:

```
HandOverActivity
HandOverActivityItem
HandOverWorkSpaceActivityItems
HandOverDefects
HandOver_WorkActivity
HandOver_WorkActivityItems
```

Also index:

```
SOPs
inspection procedures
defect descriptions
handover documentation
status definitions
workflow rules
```

Then questions such as:

```
What defects were found during the handover?

What activities are still pending?

What is the procedure for closing a defect?

Which workspace has the highest number of defects?

Explain the handover workflow.
```

Now you're learning RAG **and building something useful**.

Phase 7 — LangGraph

Duration: Week 6–7

This should be your most important framework after LangChain fundamentals.

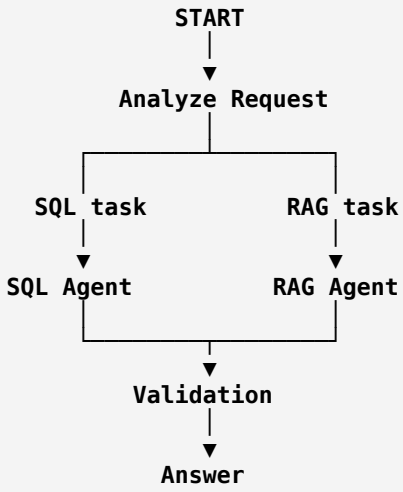
Don't just learn:

```
Python
create_agent()
```

Learn what happens underneath.

The course explicitly walks through manually implementing the agent loop before moving toward LangGraph, which is excellent for understanding the architecture rather than treating the framework as magic. Udemy

Your mental model should become:



LangGraph is where your application starts becoming an **orchestrated workflow**, rather than merely an LLM wrapper.

Phase 8 — MCP + Production Agents

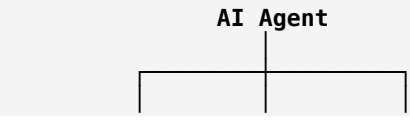
Duration: Week 7–8

The current course includes MCP and production-oriented agentic architecture. Udemy

Learn:

```
MCP
Tools
Resources
Prompts
Servers
Clients
```

Then consider:



```
          ▼       ▼       ▼
        MCP     MCP     MCP
        SQL     Qdrant  Search
      Server    Server  Server
```

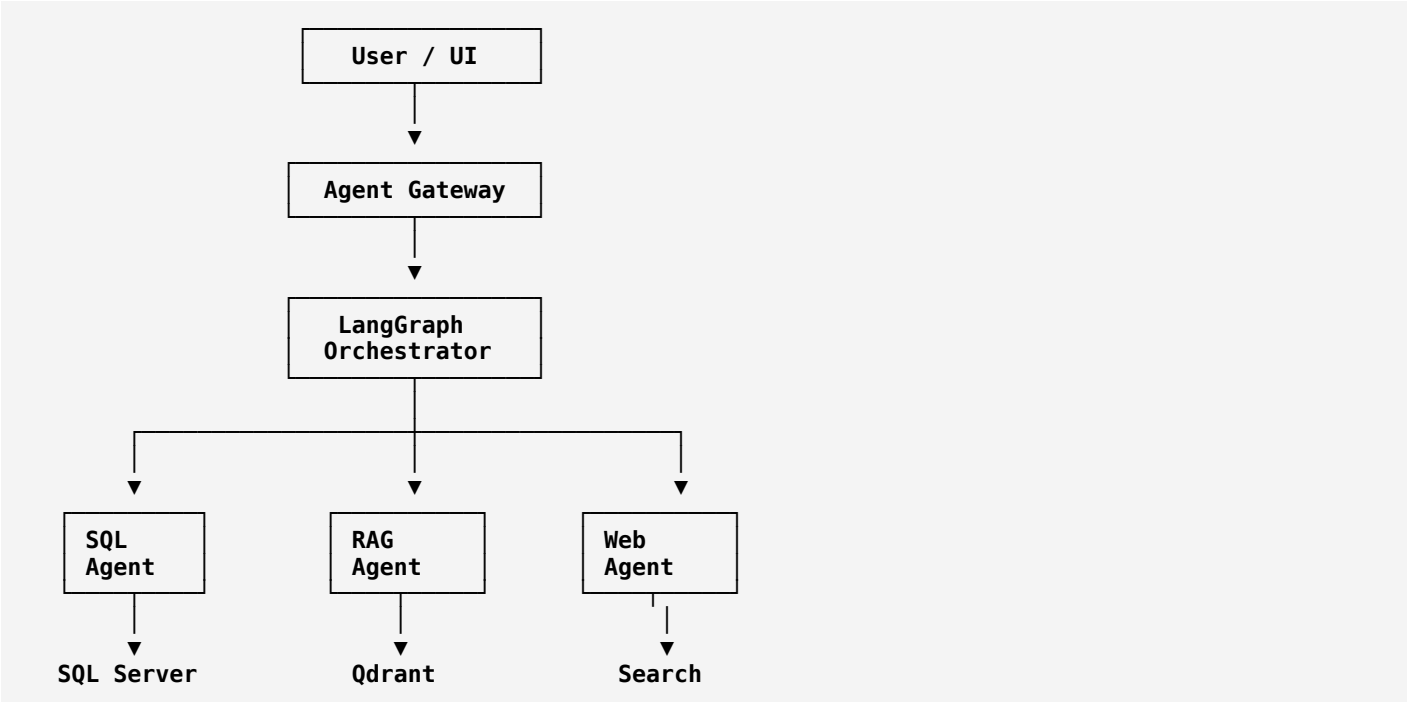
This is particularly interesting for your architecture because it lets you decouple capabilities from the agent.

Final Project

After completing the course, I would **not** stop.

Build this:

Intelligent Enterprise AI Agent



With:

```
Ollama / Cloud LLM
        │
        ▼
   LangChain
        │
        ▼
   LangGraph
        │
        ├── Tools
        ├── RAG
        ├── MCP
        ├── Memory
        └── Workflows
        │
        ▼
   LangSmith
```

Your 8-Week Schedule

Week	Focus	Deliverable
1	Python + LangChain fundamentals	Basic LLM chains
2	LCEL + Ollama + structured output	SQL explanation service
3	Tools + agents + function calling	Database agent
4	RAG fundamentals	Qdrant RAG
5	Advanced RAG	Property Handover Assistant
6	Agent architecture + LangGraph	Multi-step agent
7	MCP + multi-agent concepts	MCP-enabled tools
8	Production architecture	Enterprise AI Agent

What I Would Skip / Deprioritize

You don't need to spend equal time on everything.

🟢 Master

```
LangChain fundamentals
LCEL
Chat models
Structured output
Tool calling
Agents
RAG
Embeddings
Vector databases
LangGraph
MCP
LangSmith
Evaluation
Production architecture
```

🟡 Understand

```
ReAct internals
Prompt engineering
Context engineering
Memory
Multi-agent systems
Deep agents
```

🔴 Don't obsess over

```
Old LangChain APIs
Legacy chains
Copying instructor projects verbatim
Pinecone-specific implementation details
Streamlit UI
```

The reason is that the current course itself has been updated for LangChain 1.2+, and the framework's APIs have evolved considerably. Udemy

---

Most Important Change I'd Make for You

Don't follow the course project-by-project.

Follow it:

```
COURSE CONCEPT
      ↓
UNDERSTAND
      ↓
REIMPLEMENT
      ↓
APPLY TO YOUR DOMAIN
      ↓
COMMIT TO GITHUB
```

For example:

Instructor teaches

Search Agent

You build

Real Estate Research Agent

Instructor teaches

Documentation RAG

You build

Property Handover RAG

Instructor teaches

Tool calling

You build

SQL Server Tool Agent

Instructor teaches

Agent

You build

Intelligent DB Agent

Instructor teaches

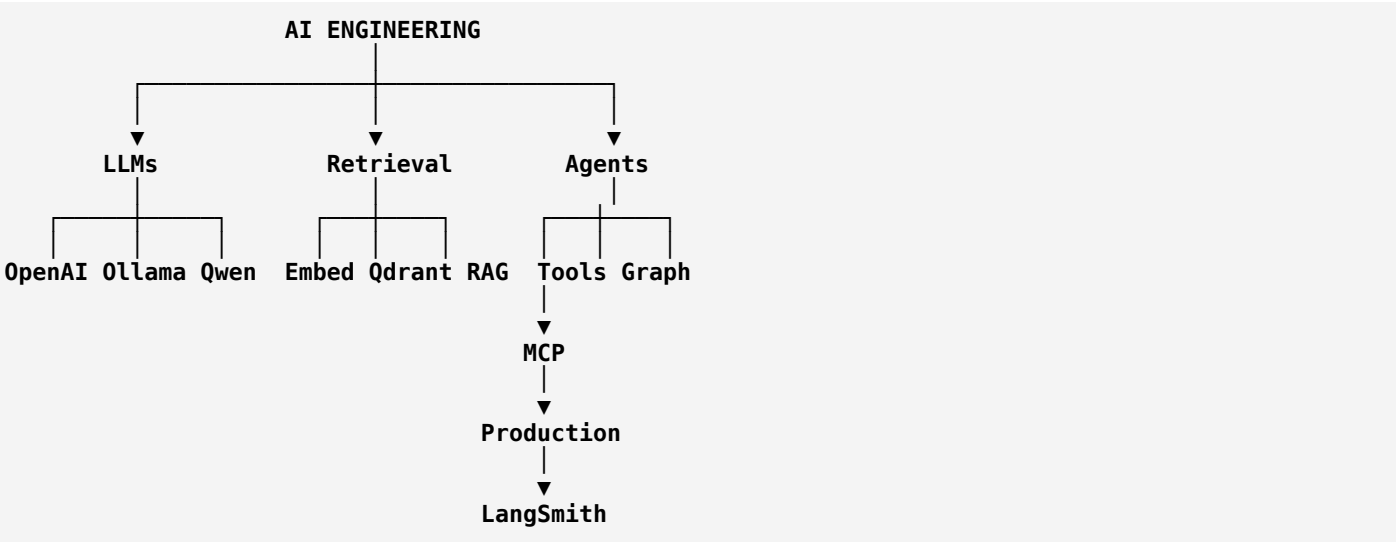
LangGraph

You build

SQL + RAG + Validation LangGraph

Your Target Skill Stack

By the end, I would want you to be comfortable with this:



And because you're already strong in **.NET + SQL Server**, I'd deliberately make your architecture:

```
.NET 8
  │
  │ REST / gRPC
  ▼
Python AI Service
  │
  ├── LangChain
  ├── LangGraph
  ├── Qdrant
  ├── Ollama
  └── LangSmith
```

rather than trying to force LangChain into your .NET backend.

That gives you a very strong separation:

```
.NET
= Enterprise application / APIs / auth / business logic

Python
= AI / agents / RAG / orchestration

SQL Server
= transactional data

Qdrant
= semantic retrieval

Ollama / Cloud
= LLM

LangGraph
```

```
= agent workflow

LangSmith
= observability/evaluation
```

Bottom line

I would take this course. 9/10 fit for your goals.

But I'd take it with one rule:

> **Every major course concept must become a component of your own Intelligent Enterprise AI Agent.**

That approach will turn the ~20-hour course into a **2-month practical AI-engineering roadmap**, rather than just another Udemy certificate.

ChatGPT can make mistakes. Check important info.